WEEK 2 · LESSON 11 OF 24 · HANDOUT

Naive Bayes: Fast Probability Classifiers

Study time: **about 40 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=Q8l0Vip5YUw — open the lesson online to watch inside the course.

🎓 **Go deeper:** Harvard CS50 AI, Lecture 2: Uncertainty (probability and Bayes) — https://www.youtube.com/watch?v=D8RRq3TbtHU

Your goal in this lesson: Explain how Naive Bayes uses Bayes' theorem with a "naive" independence assumption, and build a small SMS spam filter with scikit-learn.

Bayes' theorem again

In Math & Python for AI you used Bayes' theorem to update a belief with evidence. **Naive Bayes** turns that into a classifier:

> $P(spam \mid words)$ is proportional to $P(spam) \times P(word_1 \mid spam) \times P(word_2 \mid spam) \times \ldots$

- **P(spam)** — how common spam is overall (the prior).
- **P(word | spam)** — how often each word appears in spam messages, learned from the training data.

The model computes this score for every class (spam and not spam) and picks the larger one.

Why "naive"?

It assumes every word (feature) is **independent** of the others once you know the class. That is clearly false — "free" and "prize" often appear together. Surprisingly, Naive Bayes still works well, especially for **text**, because it only needs the **right ranking** of classes, not perfect probabilities.

A tiny SMS spam filter

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

messages = [
    "Win a free phone now", "Claim your prize money today",
    "Meeting moved to 3pm", "Can you send the report",
    "Free airtime click this link", "Lunch at noon tomorrow?",
    "You won cash prize claim now", "Please review the budget file",
]
labels = [1, 1, 0, 0, 1, 0, 1, 0]          # 1 = spam

vec = CountVectorizer()                     # turns text into word counts
X_text = vec.fit_transform(messages)
print(vec.get_feature_names_out()[:10])

spam_filter = MultinomialNB().fit(X_text, labels)

new = ["Free prize waiting claim now", "Send me the meeting notes"]
print(spam_filter.predict(vec.transform(new)))
print(spam_filter.predict_proba(vec.transform(new)).round(3))
```

The first new message is classified as spam (probability about **0.99**), the second as normal (spam probability about **0.07**) — after seeing only eight examples!

`CountVectorizer` builds a **bag of words**: one column per word, counting how often it appears. Word order is ignored.

Smoothing: the zero problem

If the word "airtime" never appeared in normal training messages, P(airtime | normal) would be 0 and would wipe out the whole product. Naive Bayes adds a small count to every word (**Laplace smoothing**, the `alpha=1.0` setting) so nothing is ever exactly zero.

Naive Bayes for numbers

`GaussianNB` assumes each numeric feature follows a normal (bell) curve within each class.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 2000
income = rng.lognormal(12.2, 0.45, n).round(-3)
loan = rng.uniform(20_000, 300_000, n).round(-3)
months = rng.integers(1, 61, n)
late = rng.poisson(0.6, n)
salary = rng.random(n) < 0.55
z = -3.2 + 2.5 * (loan / income) - 0.03 * months + 0.8 * late - 0.9 * salary
default = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int)
loans = pd.DataFrame({"income_rwf": income, "loan_rwf": loan, "months_customer": months,
                      "late_payments": late, "salaried": salary.astype(int),
                      "defaulted": default})
print(loans.shape, "default rate:", round(loans["defaulted"].mean(), 3))
loans.head()
```

```
from sklearn.model_selection import train_test_split

X = loans.drop(columns="defaulted")
y = loans["defaulted"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
```

```
from sklearn.naive_bayes import GaussianNB

nb = GaussianNB().fit(X_train, y_train)
print("Test accuracy:", round(nb.score(X_test, y_test), 3))
```

About **0.81** — lower than logistic regression (0.84), because income and loan size are not normally distributed and the model cannot combine them into the loan-to-income ratio that really drives risk.

Which Naive Bayes?

Version	Features	Typical use
MultinomialNB	Word counts	Spam filters, topic classification
BernoulliNB	Yes/no features (word present or not)	Short texts
GaussianNB	Numbers	Quick baseline on numeric data

Strengths and weaknesses

Naive Bayes trains in a blink, needs little data, and works well for text. Its probabilities are often too extreme (close to 0 or 1), and it cannot learn interactions between features.

Key terms

- **Naive Bayes** — a classifier using Bayes' theorem with independent features.
- **Prior** — how common a class is before seeing the evidence.
- **Bag of words** — text represented by word counts, ignoring order.
- **Laplace smoothing** — adding a small count so no probability is zero.

Remember this: Naive Bayes multiplies simple word (or feature) probabilities; it is a fast, strong baseline for text even though its independence assumption is false.

Name: _____ Date: _____

Activity

Task 1

Add five more messages of your own (some in Kinyarwanda, for example "Watsindiye amafaranga") and retrain. Does it still work?

Task 2

Try `BernoulliNB` on the same messages.

Task 3

Print the words with the biggest difference in `spam_filter.feature_log_prob_` between the two classes. Which words are most "spammy"?

Task 4

Explain to a friend why the model is called "naive".

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Which theorem is Naive Bayes built on?
   - A. Fermat's theorem
   - B. Pythagoras
   - C. Bayes' theorem
   - D. The central limit theorem
2. What is 'naive' about Naive Bayes?
   - A. It assumes features are independent given the class
   - B. It uses little data
   - C. It has no prior
   - D. It cannot use text
3. What does CountVectorizer do?
   - A. Translates text
   - B. Turns text into word-count columns (bag of words)
   - C. Removes spam
   - D. Scales numbers
4. Why is Laplace smoothing used?
   - A. To remove stop words
   - B. To scale features
   - C. To stop a never-seen word from making a probability exactly zero
   - D. To speed up training
5. Which version suits word counts?
   - A. MultinomialNB
   - B. KMeans
   - C. LinearRegression
   - D. GaussianNB
6. Why did GaussianNB do worse than logistic regression on the loans?
   - A. The data has no labels
   - B. It overfits deep trees
   - C. It cannot combine features such as loan and income, and the features are not bell-shaped
   - D. It needs more trees

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.